Noise & Restoration

Week 4 — Modelling and removing noise

Practical Image Processing

Lecture 4 ▪ 60–70 min

Recap & today

- Wk3: filters combine a pixel with its neighbours (convolution).
- Today: **noise** — where it comes from, how to **model** it, and how to **remove** it without destroying the detail you need.

Why it matters

Real meter photos are noisy (low light, high ISO). Denoising is what makes the digits thresholdable and readable.

Roadmap

Noise models

Restoration

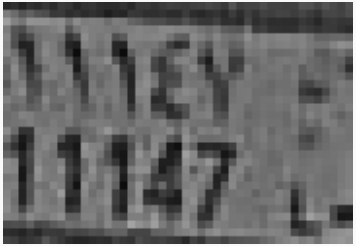
In-class exercise

Deep dive

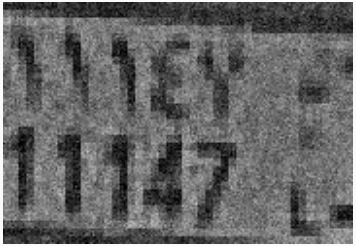
Wrap-up

Noise models

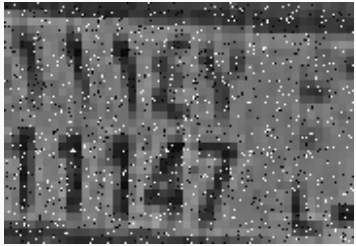
Two noises you must recognize



clean



Gaussian



salt-and-pepper

Key idea

The noise *type* decides the *cure*. Gaussian $\rightarrow$ averaging; impulse $\rightarrow$ median. Diagnose first.

Where noise comes from (physics, Week 1)

- **Gaussian noise**: small random fluctuations added to every pixel (sensor/read noise).
- **Salt-and-pepper** (impulse): random pixels stuck at 0 or 255 (dead pixels, transmission).
- **Poisson** (shot) noise: from photon counting; stronger in dark areas / low light.

Math

Gaussian model: $g = f + n$, with $n \sim \mathcal{N}(0, \sigma^2)$ — additive and zero-mean.

Diagnosing noise

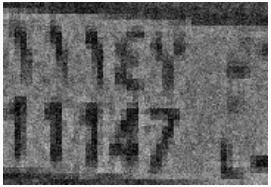
- Look at a **flat** region: Gaussian = a fuzzy spread; salt-pepper = isolated black/white specks.
- The histogram of a flat patch: Gaussian widens the peak; impulse adds spikes at 0 and 255.
- Match the filter to what you see — using the wrong one wastes detail or fails to clean.

✕ Pitfall

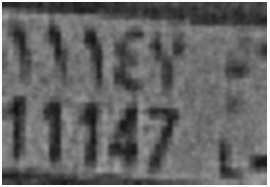
Averaging (Gaussian blur) barely touches salt-and-pepper — it just spreads each speck into a gray smudge.

Restoration

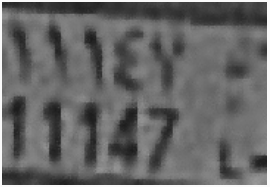
Removing Gaussian noise



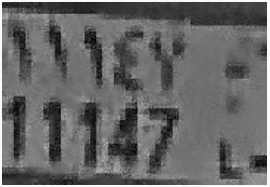
noisy



Gaussian blur



bilateral



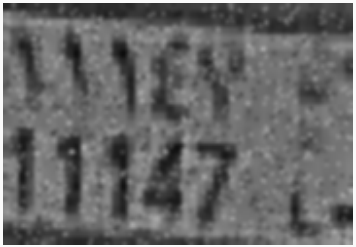
non-local means

Gaussian blur smooths but blurs edges; bilateral keeps edges; NLM is best but slowest.

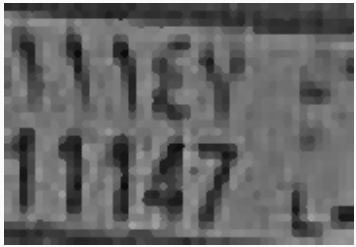
Removing salt-and-pepper — median wins



noisy



Gaussian (bad)



median (great)

Key idea

The median ignores outliers: a speck of 255 among 100s doesn't move the median. This is why median crushes impulse noise while keeping edges.

Why median beats averaging for impulse noise

- Mean of $\{100, 100, 100, 255\} = 164$ — the outlier drags it up.
- Median of the same $= 100$ — the outlier is ignored entirely.
- Averaging *spreads* an impulse; median *deletes* it. Match filter to noise.

Math

Median is robust to outliers; the mean is not. That single fact explains the whole result.

Bilateral & non-local means

- **Bilateral**: average only neighbours of *similar intensity* → smooth flats, keep edges.
- **Non-local means (NLM)**: average pixels with a similar *neighbourhood pattern*, anywhere in the image.
- NLM gives the best quality on Gaussian noise, at a real speed cost.

✓ Good practice

For DocScan: bilateral or NLM before thresholding keeps digit edges crisp while killing sensor grain.

A quick reference of noise → filter

Noise	Looks like	Best filter
Gaussian	fuzzy grain everywhere	Gaussian / NLM
Salt-and-pepper	isolated black/white dots	median
Poisson (shot)	grain, worse in dark	NLM / variance-stabilize
Periodic	repeating wave/pattern	frequency-domain (Week 6)

Key idea

Periodic noise is invisible to spatial filters — you remove it in the frequency domain (Week 6).

Denoising in code

```
gauss     = cv2.GaussianBlur(noisy, (5,5), 0)
median    = cv2.medianBlur(noisy, 3)              # salt-and-pepper
bilateral = cv2.bilateralFilter(noisy, 9, 60, 60)  # edge-preserving
nlm       = cv2.fastNlMeansDenoising(noisy, None, 15, 7, 21)  # best, slow
```

Note

Try median first for specks; bilateral/NLM when you must keep edges. Measure, don't guess.

In-class exercise

In-class exercise (10 min)

Pick the filter

For each situation, which filter and why?

1. A scanned meter with random black/white dots.
2. A low-light phone photo with fine grain everywhere.
3. A document where you must keep the text edges razor-sharp while removing grain.
4. How would you *measure* whether your denoising actually helped?

Debrief

- (1) Salt-and-pepper → **median**. (2) Gaussian grain → Gaussian/NLM.
- (3) Edge-preserving → **bilateral** (or NLM). (4) Measure with **PSNR**/SSIM vs a clean reference, or by downstream OCR accuracy.
- Golden rule: the **cheapest** filter that removes the noise *your* pipeline cares about.

Key idea

"Denoise" isn't one button — it's choosing the model that matches the noise.

Deep dive

Measuring restoration: PSNR & SSIM

- **PSNR**: based on mean-squared error vs a clean reference; higher = closer (in dB).
- **SSIM**: perceptual — compares structure, not just pixel errors; better matches human judgment.
- If you have a clean reference (or a synthetic-noise experiment), *measure*, don't eyeball.

Math

$$\mathrm{PSNR} = 10 \log_{10} \frac{255^2}{\mathrm{MSE}}$$

The denoising dilemma

- Every denoiser trades **noise removal** against **detail preservation**.
- Remove too little $\rightarrow$ noise breaks the next stage; too much $\rightarrow$ you erase the digits.
- There is no free lunch: choose the operating point for *your* goal (readable digits).

Key idea

Denoise for the downstream task, measured on hard images — not for the prettiest flat wall.

Temporal denoising: average many frames

- If you have several shots of the *same* static scene, average them: noise cancels, signal stays.
- N frames reduce Gaussian noise by $\sqrt{N}$ — the cheapest, highest-quality denoiser.
- For a meter you can photograph twice, this beats any single-image filter.

✓ Good practice

When you control capture, averaging a burst is the best denoising you can do — no detail lost.

Restoration vs enhancement

- **Enhancement** (Wk2) makes an image *look* better (subjective).
- **Restoration** tries to *recover* the original by modelling the degradation (objective).
- Deblurring is restoration too: if you know the blur (the **PSF**), you can partly invert it.

Deblurring & the PSF (brief)

- Blur = convolution with a **point-spread function** (PSF). Deblur = deconvolution.
- Naive inversion amplifies noise catastrophically; real methods (Wiener, Richardson-Lucy) regularize.
- Motion/defocus blur on a meter photo is often better *avoided* (steady shot) than fixed.

✕ Pitfall

Perfect deblurring is impossible when information is truly lost. Prevention beats restoration.

Where denoising sits in the pipeline



- Denoise *early* — before enhancement (which would amplify the noise) and before thresholding (which noise would corrupt).
- But not *too* aggressively: over-denoising erases the digits you need.

Adaptive & order-statistic filters

- **Adaptive median**: grows its window only where noise is detected — cleans heavy impulse noise with less blur.
- **Min/Max** filters: remove pepper (min) or salt (max) specifically.
- These are the "order-statistic" family — like median, they pick a ranked value, not an average.

✔ Good practice

For very noisy meter scans, an adaptive median can save digits a plain median would smear.

Deep learning denoisers (preview)

- Modern denoisers are CNNs trained on (noisy, clean) pairs — they beat classical methods.
- But they need data, compute, and can *hallucinate* detail that isn't there.
- For DocScan, classical (median/bilateral/NLM) is usually enough and fully interpretable.

Wrap-up

What you should now be able to do

- Recognize Gaussian vs impulse noise and name their physical origins.
- Choose the matching filter (median / Gaussian / bilateral / NLM).
- Measure restoration with PSNR/SSIM instead of eyeballing.
- Reason about the noise-vs-detail trade-off for your pipeline.

This week's deliverable

Due before Week 5 (see `project/`)

1. Add Gaussian and salt-pepper noise to a clean meter image (known noise = ground truth).
2. Denoise each with 3 filters; compute PSNR vs the clean original; tabulate.
3. Show a case where the "wrong" filter fails (Gaussian on salt-pepper).
4. DocScan: denoise a real noisy meter photo so its Otsu/adaptive binary is clean.

Match the filter to the noise.

Median for specks, averaging for grain, bilateral to keep edges.